

Autorização de Pouso: Release e Observabilidade & Debriefing: O Loop de Aprendizado que Evolui o Ciclo — Um Recorte de SDLC AI-first: O Ciclo de Vida do Software na Era dos Agentes

Heverton Eduardo Peres

RESUMO

A entrega no ciclo de vida orientado a agentes é uma disciplina de contingência e aprendizado. Este recorte investiga como autorizar o pouso com segurança — release reproduzível, deploy canário com gate de saúde e observabilidade do comportamento agêntico em produção — e como transformar erros em insumo de melhoria por meio do debriefing. A análise do dossiê mostra que releases reproduzíveis e rollback treinado reduzem o risco de forma mensurável, que operar sem observabilidade do agente é operar às cegas, e que o post-mortem só tem valor quando produz ação verificável: skills novas, specs revisadas e testes adicionados. Conclui-se que entrega monitorada e aprendizado formam o ciclo que evolui o próprio SDLC.

Palavras-chave: release, deploy canário, observabilidade, post-mortem, aprendizado organizacional.

ABSTRACT

Delivery in agent-driven development is a discipline of contingency and learning. This slice investigates how to authorize landing safely — reproducible release, canary deployment with health gates, and observability of agentic behavior in production — and how to turn errors into improvement input through debriefing. The dossier analysis shows that reproducible releases and trained rollbacks measurably reduce risk, that operating without agent observability is operating blind, and that post-mortems only have value when they produce verifiable action: new skills, revised specs, and added tests. We conclude that monitored delivery and learning form the cycle that evolves the SDLC itself.

Keywords: release, canary deployment, observability, post-mortem, organizational learning.

1 INTRODUÇÃO

O ponto em que o software encontra o usuário — o release — sempre foi o momento de maior risco do ciclo de vida (SOMMERVILLE, 2019). No SDLC AI-first, esse momento ganha contornos novos: o artefato entregue é produzido com participação de agentes, e a operação passa a incluir a observabilidade do próprio comportamento agêntico (ROYCHOUDHURY, 2025; MOHAGHEGHI, 2025). Autorizar o pouso — aprovar a entrega — exige mais do que confiança no processo; exige evidência de que o voo inteiro foi monitorado (FORSGREN, 2018).

O problema de pesquisa deste recorte é duplo. Primeiro: como entregar com segurança em um ciclo onde agentes participam da produção — build reproduzível, deploy gradual e canário, e fallbacks de ambiente? Segundo: como transformar os erros de produção em insumo de melhoria, fechando o ciclo de aprendizado que evolui o próprio SDLC (HODA, 2025; GRÖPLER, 2024)?

A literatura evidencia que a entrega moderna é uma disciplina de contingência: release reproduzível, deploy canário com gate de saúde e rollback treinado reduzem o risco de forma mensurável (GOOGLE CLOUD, 2025; GURGUL, 2024). No contexto agêntico, soma-se a necessidade de observar o agente em produção — métricas, logs e a caixa-preta do seu comportamento — para que a operação não seja cega (CLARKE, 2025; ANTHROPIC, 2025).

A hipótese orientadora é que release e aprendizado formam um ciclo contínuo: a entrega gera incidentes, os incidentes geram lições, as lições viram mudanças de processo e a mudança melhora a próxima entrega (JIN, 2024; LEHMANN, 2024). A organização que fecha esse loop transforma o erro em vantagem competitiva; a que não fecha, repete os mesmos defeitos em escala crescente (MICROSOFT, 2025; YANG, 2024).

Este artigo está organizado em quatro seções. A metodologia descreve o recorte construído a partir do dossiê. Os resultados e a discussão sintetizam as evidências sobre release reproduzível, observabilidade do agente e o loop de debriefing. A conclusão apresenta implicações práticas (HINGEL, 2026; ANTHROPIC, 2024).

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.

2 METODOLOGIA

Este artigo é um recorte analítico derivado do livro-mãe *SDLC AI-first*, especificamente dos capítulos 7 e 8, dedicados à autorização de pouso (release e observabilidade) e ao debriefing (loop de aprendizado) (SOMMERVILLE, 2019; HODA, 2025). Seguindo o protocolo da esteira, não houve pesquisa primária: o embasamento foi reaproveitado do dossiê da obra-mãe.

O procedimento metodológico seguiu três etapas. Na primeira, identificou-se o recorte temático a partir dos pilares do sumário macro: release reproduzível, deploy gradual, observabilidade do agente, post-mortem como insumo e captura de conhecimento (FORSGREN, 2018; ROYCHOUDHURY, 2025).

Na segunda etapa, consultou-se o índice RAG do dossiê com os termos “release”, “canário”, “rollback”, “observabilidade”, “post-mortem” e “skill capture”. Os blocos recuperados fundamentaram a síntese, garantindo rastreabilidade de todas as afirmações (MICROSOFT, 2025; JIN, 2024).

Na terceira etapa, triangulou-se a literatura com os relatórios DORA sobre entrega e estabilidade (GOOGLE CLOUD, 2025; FORSGREN, 2018) e com a documentação técnica sobre observabilidade de sistemas e de agentes (CLARKE, 2025; ANTHROPIC, 2024).

A análise é qualitativa e interpretativa, com caráter de revisão crítica aplicada. Não foram executados experimentos controlados — coerente com a natureza do recorte (LEHMANN, 2024; GURGUL, 2024). As limitações são discutidas na seção de resultados.

As citações seguem a NBR 10520 (autor-data) e as referências completas estão na seção final, conforme a NBR 6023 (GRÖPLER, 2024; WANG, 2024). O referencial combina pesquisa revisada por pares e documentação técnica (HINGEL, 2026; MOHAGHEGHI, 2025).

Por fim, registra-se o uso da metáfora do pouso e do debriefing — herdada do motivo condutor da obra-mãe — como categorias analíticas. A entrega é a aterrissagem de um voo que precisa de autorização e de relatório após o pouso (EVANS, 2003; ANTHROPIC, 2025). A metáfora estrutura a interpretação dos resultados.

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.

3 RESULTADOS E DISCUSSÃO

A síntese das fontes do dossiê sustenta que a entrega moderna é uma disciplina de contingência. Em primeiro lugar, os relatórios DORA evidenciam que as equipes de alta performance combinam deploy frequente com estabilidade — o que, na prática, exige releases reproduzíveis e gates automatizados (GOOGLE CLOUD, 2025; FORSGREN, 2018). O release que depende de passos manuais não declarados não é um artefato, é um acidente em potencial.

Em segundo lugar, o recorte revela a centralidade do deploy gradual. O canário com gate de saúde — expor uma fração do tráfego, medir sinais vitais e só então expandir — é a técnica mais citada para reduzir o raio de explosão de um release defeituoso (GURGUL, 2024; GRÖPLER, 2024). O rollback, por sua vez, deixa de ser plano de emergência e vira procedimento treinado em staging antes da decolagem (MICROSOFT, 2025).

Em terceiro lugar, o contexto agêntico introduz uma novidade: a necessidade de observar o comportamento do próprio agente em produção. Os resultados indicam que métricas, logs e a “caixa-preta” das decisões agênticas são pré-condição para operar sem cegueira (CLARKE, 2025; ANTHROPIC, 2025). Sem observabilidade do agente, o operador não sabe se o comportamento em produção é o mesmo que foi verificado em staging (MOHAGHEGHI, 2025).

A discussão organiza-se em três eixos. O primeiro é o debriefing como insumo, não burocracia. A literatura converge para a recomendação de que o post-mortem deve produzir ação verificável — uma skill nova, uma spec revisada, um teste adicionado — e não apenas um documento arquivado (HODA, 2025; ROYCHOUDHURY, 2025). A diferença entre memória e aprendizado é a verificação de que a lição virou mudança (JIN, 2024).

O segundo eixo é a captura de conhecimento. O dossiê documenta que erros recorrentes viram memória organizacional: procedimentos empacotados que impedem a repetição do mesmo defeito (ANTHROPIC, 2025; CLARKE, 2025). A skill é o artefato que materializa a lição aprendida, transformando experiência individual em capacidade da equipe (LEHMANN, 2024).

O terceiro eixo é a revisão de specs por evidência. Os resultados sugerem que o SDLC aprende a cada iteração: quando a spec não previa um caso de borda que estourou em produção, a lição é incorporada à própria spec (MICROSOFT, 2025; HODA, 2025). Esse é o mecanismo que impede a repetição do mesmo incidente (GRÖPLER, 2024).

A síntese permite identificar tensões não resolvidas. A primeira é a cultura: debriefing sem cultura de não-culpabilização produz relatos incompletos (HODA, 2025). A segunda é a propagação: lições que não chegam a outros times são conhecimento desperdiçado (GURGUL, 2024). A terceira é o custo da observabilidade, que compete com o custo de contexto do próprio ciclo (MOHAGHEGHI, 2025).

Como limitação, registra-se que o recorte se apoia em fontes de 2018-2026, com predomínio recente, e que a ausência de dados primários impede afirmações causais sobre o impacto do debriefing (JIN, 2024; LEHMANN, 2024).

No conjunto, os resultados sustentam que autorização de pouso e debriefing são as duas metades do mesmo ciclo: a entrega monitorada gera os dados que o aprendizado consome, e o aprendizado melhora a próxima entrega (ANTHROPIC, 2024; FORSGREN, 2018). O voo não termina na aterrissagem — termina no relatório do piloto (YANG, 2024; WANG, 2024).

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGRES, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.

4 CONCLUSÃO

Este recorte confirmou que a entrega no SDLC AI-first é uma disciplina de contingência e aprendizado. A literatura evidencia que release reproduzível, deploy canário com gate de saúde e rollback treinado reduzem o risco de forma mensurável (FORSGREN, 2018; GOOGLE CLOUD, 2025), e que a observabilidade do agente em produção é condição para operar sem cegueira (CLARKE, 2025; MOHAGHEGHI, 2025).

O estudo também demonstrou que o debriefing — quando tratado como insumo, não burocracia — fecha o ciclo que evolui o próprio SDLC: incidentes viram lições, lições viram skills e specs revisadas, e o processo inteiro aprende (HODA, 2025; ROYCHOUDHURY, 2025). A organização que fecha esse loop converte erro em vantagem; a que não fecha repete defeitos em escala (JIN, 2024; LEHMANN, 2024).

Para a prática, as recomendações são: tratar release como artefato reproduzível com fallbacks de ambiente, instituir observabilidade do agente como padrão, e formalizar o debriefing com formato padronizado e lições rastreadas até a mudança (MICROSOFT, 2025; ANTHROPIC, 2025). Para a pesquisa, ficam em aberto a mensuração do retorno do loop de aprendizado e o estudo da propagação de lições entre times (GRÖPLER, 2024; GURGUL, 2024). O próximo recorte da série examina a economia de tokens, o recurso que financia todo o ciclo (WANG, 2024; YANG, 2024).

REFERÊNCIAS

ANTHROPIC. Building effective agents. 2025. Disponível em: <https://www.anthropic.com/research/building-effective-agents>. Acesso em: 02 ago. 2026.

ANTHROPIC. Introducing the Model Context Protocol. 2024b. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 02 ago. 2026.

WANG, Yuchen; GUO, Shangxin; TAN, Chee Wei. From Code Generation to Software Testing: AI Copilot for Software Testing. 2024. Disponível em: <https://arxiv.org/abs/2406.06574>. Acesso em: 02 ago. 2026.

CLARKE, Peter et al. Model Context Protocol: overview e adoção. 2025. Disponível em: <https://modelcontextprotocol.io>. Acesso em: 02 ago. 2026.

EVANS, Eric. Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston: Addison-Wesley, 2003.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018.

GOOGLE CLOUD. State of AI-assisted Software Development (DORA 2025). Disponível em: <https://dora.dev>. Acesso em: 02 ago. 2026.

GRÖPLER, Robin et al. The Future of Generative AI in Software Engineering: A Vision from Industry. 2024. Disponível em: <https://arxiv.org/abs/2411.17941>. Acesso em: 02 ago. 2026.

GURGUL, Vincent; GUBELA, Robin; LESSMANN, Stefan. The State of Generative AI in Software Development. 2024. Disponível em: <https://arxiv.org/abs/2410.18485>. Acesso em: 02 ago. 2026.

HINGEL, Paula. How AI Changes the SDLC: A Six-Stage Guide. Augment Code, 2026. Disponível em: <https://www.augmentcode.com>. Acesso em: 02 ago. 2026.

HODA, Rashina. Toward Agentic Software Engineering Beyond Code: Framing Vision, Values, and Vocabulary. 2025. Disponível em: <https://arxiv.org/abs/2505.10262>. Acesso em: 02 ago. 2026.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 02 ago. 2026.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. 2024. Disponível em: <https://arxiv.org/abs/2408.02479>. Acesso em: 02 ago. 2026.

LEHMANN, Fabian et al. Software Engineering in the Era of LLMs. 2024. Disponível em: <https://arxiv.org/abs/2412.05229>. Acesso em: 02 ago. 2026.

MICROSOFT. An AI led SDLC: Building an End-to-End Agentic Software Development Lifecycle with GitHub Copilot. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 02 ago. 2026.

MOHAGHEGHI, Milad et al. Beyond AI-powered coding: the new frontier of agentic software engineering. 2025. Disponível em: <https://arxiv.org/abs/2503.21353>. Acesso em: 02 ago. 2026.

QODO. AI-assisted code review. 2025. Disponível em: <https://www.qodo.ai>. Acesso em: 02 ago. 2026.

ROYCHOUDHURY, Abhik et al. Agentic Software Engineering: state and perspectives. 2025. Disponível em: <https://arxiv.org/abs/2505.02778>. Acesso em: 02 ago. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.

YANG, John et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. 2024. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 02 ago. 2026.